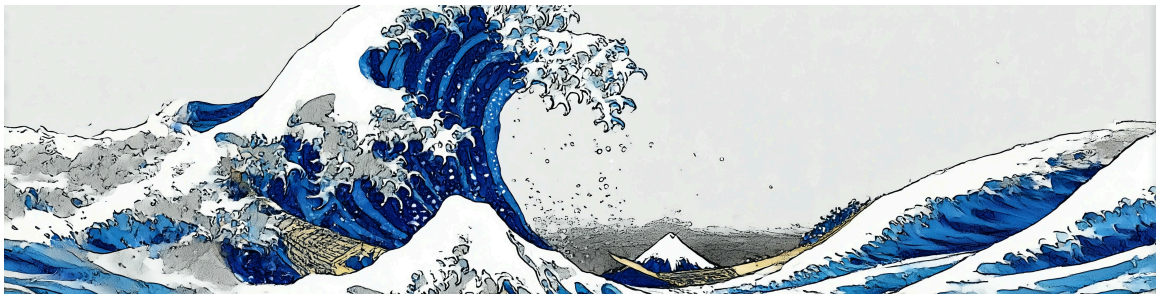
Unit-Tests

Joern Ploennigs · AI4SC



> The only way to go fast is to go well.
>
> — Robert C. Martin

Unit-Tests

Unit-Tests sind zum Vermeiden und Diagnostizieren von Fehlern sehr wichtig. Ein Unit-Test testet ein einzelnes Code-Modul, wie z.B. eine Funktion oder eine Klasse.

In der agentischen Programmentwicklung sind Tests mehr als Fehlererkennung — sie sind die **Spezifikation**: Sie definieren das erwartete Verhalten, bevor die KI Code generiert. Ein Test, der auf KI-generierten Code anschlägt, ist kein Misserfolg — es ist der Nachweis, dass die Verifikation funktioniert.

Zum Testen verwendet man in Python meist spezielle Pakete wie `unittest` oder `nosetest`, welche dann automatisch bei Code-Änderungen ausgeführt werden. Einfache Tests können direkt in den Code geschrieben werden. Hierfür ist der `assert` Befehl da. Er prüft, ob eine Testbedingung erfüllt ist (assert) und löst eine `AssertionError` Exception aus, wenn dies nicht der Fall ist.

Wir wollen zum Beispiel die zuvor definierte Divisionsfunktion `division()` testen.

```
def division(zaehler, nenner):
    if not isinstance(nenner, (int, float)):
        raise ValueError(f"Nenner nicht vom Datentyp `int` oder `float`")
    elif not isinstance(zaehler, (int, float)):
        raise ValueError(f"Zaehler nicht vom Datentyp `int` oder `float`")
    elif not nenner:
        print("Warnung Division durch 0")
        return None
    else:
```

```
        ergebnis = zaehler / nenner
        print(f"Das Ergebnis von {zaehler}/{nenner} = {ergebnis}")
        return ergebnis
```

Test-Driven Prompting

In der agentischen Programmentwicklung verändern Tests ihre Rolle: Sie sind nicht mehr nur Prüfinstrument für fertigen Code, sondern **Spezifikation** — die präzise Beschreibung des gewünschten Verhaltens. Wer Tests schreibt, bevor er die KI promptet, operationalisiert damit die Anforderung: aus vager Beschreibung wird messbare Bedingung.

Test-Driven Prompting (TDP) folgt dem agentischen Zyklus:

1. Anforderung verstehen
2. Tests schreiben, die das gewünschte Verhalten operationalisieren
3. KI prompten — die Tests als Akzeptanzkriterium übergeben
4. KI generiert Code
5. Tests ausführen
6. Scheitern Tests ⟶ Diagnose, Reprompt, iterieren

Ein fehlgeschlagener Test auf KI-generierten Code ist kein Misserfolg. Er bedeutet: der Fehler wurde vor dem Einsatz gefunden.

Umwelt-Beispiel: Bodenverschmutzung klassifizieren

Als durchgehendes Beispiel für Test-Driven Prompting verwenden wir eine typische Umweltingenieur-Aufgabe: die Klassifikation von Bodenkontaminationswerten.

Anforderung: Schreibe eine Funktion `klassifiziere_boden(wert)`, die einen Messwert in mg/kg als `"Sauber"`, `"Beobachten"` oder `"Kritisch"` klassifiziert. Grenzwerte: < 50 ⟶ Sauber, 50–149 ⟶ Beobachten, ≥ 150 ⟶ Kritisch.

Bevor wir die KI prompten, schreiben wir 5 Tests: 3 Normalfälle und 2 Grenzwertfälle. Diese Tests übergeben wir als Akzeptanzkriterium im Prompt.

Testabdeckung

Selbst wenn alle geschriebenen Tests grün sind, kann KI-generierter Code ungetestete Bereiche enthalten. *Testabdeckung* (Coverage) misst, welcher Anteil des Codes durch Tests ausgeführt wurde.

Zeilenabdeckung (Line Coverage): Welche Codezeilen wurden durch Tests ausgeführt? Eine Abdeckung von 80% bedeutet, dass 20% der Zeilen nie getestet wurden — ihr Verhalten ist unbekannt.

Verzweigungsabdeckung (Branch Coverage): Wurden alle `if`/`else`-Zweige getestet? Dies ist wichtiger als reine Zeilenabdeckung, da KI-generierter Code häufig versteckte Zweige enthält, die nur bei speziellen Eingaben aktiv werden.

Funktionstest

Hierfür schreiben wir als erstes einen *Funktionstest*. Diese überprüfen die korrekte Funktion eines Moduls mit mehreren Eingaben, für die wir das korrekte Ergebnis kennen. Hier macht man meist mehrere Tests, um sicherzustellen, dass man nicht `zufällig` das richtige Ergebnis bekommt, sondern für verschiedene Kombinationen.




```
def test_funktion(): # Die übliche Notation für Tests in Python ist die
Funktionsnamen mit `test_` zu beginnen
    assert division(10, 2) == 5
    assert division(10, 5) == 2
    assert division(50, 1) == 50
    assert division(50, 5) == 10
```

```
test_funktion()
```

```
Das Ergebnis von 10/2 = 5.0
Das Ergebnis von 10/5 = 2.0
Das Ergebnis von 50/1 = 50.0
Das Ergebnis von 50/5 = 10.0
```

Grenzwerttest

Ein weiterer wichtiger Test ist der *Grenzwerttest*. Hier prüft man ob Eingaben im Grenzwertbereich korrekt behandelt werden. Zum Beispiel ob positiv oder negativ Unendlich behandelt werden oder ob die Division durch 0 behandelt wird. Wir haben oben beim Erstellen der Funktion definiert, dass bei einer Division durch 0 die Funktion den Wert `None` ausgeben soll.




```
import math # die Zahl unendlich ist in der math bibliothek definiert

def test_grenzwert():
    assert division(math.inf, 1) == math.inf
```

```
    assert division(math.inf, 100) == math.inf
    assert math.isnan(division(math.inf, math.inf)) # Das prüfen auf nan (not
a number) ist nur durch die funktion `isnan` moeglich
    assert division(1, math.inf) == 0
    assert division(100, 0) is None
```

```
test_grenzwert()
```

```
Das Ergebnis von inf/1 = inf
Das Ergebnis von inf/100 = inf
Das Ergebnis von inf/inf = nan
Das Ergebnis von 1/inf = 0.0
Warnung Division durch 0
```

Datentyp-Test

Da Python dynamische Datentypen unterstützt, ist der *Datentyp-Test* sehr wichtig. Hierbei wird getestet, ob die Funktion falsche Datentypen als Eingabe richtig behandelt.



Hierbei können wir nicht direkt `assert` nutzen da es ja unseren Code zum Absturz bringt.

```
try:
    division("falscher typ", 1)
except Exception as e:
    assert isinstance(e, ValueError)
```

Für vollständige Tests nutzen wir einen `try-except`-Block und die `else`-Anweisung. Sie wird nur dann ausgeführt, wenn im `try`-Block *keine* Exception auftritt, was im Testfall ja unerwünscht ist. In diesem Fall lösen wir selbst einen `AssertionError` aus. Ferner prüfen wir, ob die ausgelöste Exception der Funktion auch den richtigen Typ `ValueError` hat.

```
def test_datentyp():
    # wir testen den Zähler
    try:
        division("falscher typ", 1)
    except Exception as e:
        assert isinstance(e, ValueError)
    else:
```

```
        raise AssertionError("Datatype")
    # den zweiten Fall des Nenners
    try:
        division(10, "falscher typ")
    except Exception as e:
        assert isinstance(e, ValueError)
    else:
        raise AssertionError("Datatype")
```

```
test_datentyp()
```

Debugging und Fehlerbehandlung

Wenn Tests fehlschlagen, beginnt der nächste Schritt der Verifikation: das systematische Suchen nach der Ursache. Dabei hilft *Debugging*. Zugleich müssen Programme mit erwartbaren Laufzeitfehlern robust umgehen können. Python stellt dafür mit `try`, `except`, `raise`, `else` und `finally` einen strukturierten Exception-Mechanismus bereit.

Mit Tests bei Code-Änderungen Fehler erkennen

Diese Unit-Tests haben wir absichtlich als Funktion definiert, so dass wir sie nun bei Code-Änderungen immer wieder durchführen können. So kann man dann testen, ob die Implementation die gestellten Erwartungen überhaupt oder immer noch erfüllt.

Wird zum Beispiel die Divisionsfunktion durch eine triviale Implementation überschrieben.

```
def division(zaehler, nenner):
    return zaehler / nenner
```

So funktioniert immer noch unser Funktionstest. Weshalb es meistens nicht hinreichend ist, nur diesen zu implementieren oder bei der Entwicklung nur die Funktion zu prüfen.

```
test_funktion()
```

Der Grenzwerttest wird allerdings jetzt eine Ausnahme auslösen.

```
test_grenzwert()
```

```
ZeroDivisionError: division by zero
[31m---------------------------------------------------------------------------[
[39m
[31mZeroDivisionError[39m                         Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[11][39m[32m, line 1[39m
[32m----> [39m[32m1[39m [43mtest_grenzwert[49m[43m([49m[43m)
```

```
[49m

[36mCell[39m[36m [39m[32mIn[4][39m[32m, line 8[39m, in
[36mtest_grenzwert[39m[34m()[39m
[32m      6[39m [38;5;28;01massert[39;00m math.isnan(division(math.inf,
math.inf)) [38;5;66;03m# Das prüfen auf nan (not a number) ist nur durch die
funktion `isnan` moeglich[39;00m
[32m      7[39m [38;5;28;01massert[39;00m division([32m1[39m,
math.inf) == [32m0[39m
[32m----> [39m[32m8[39m [38;5;28;01massert[39;00m
[43mdivision[49m[43m([49m[32;43m100[39;49m[43m,[49m[43m
[49m[32;43m0[39;49m[43m)[49m [38;5;129;01mis[39;00m
[38;5;28;01mNone[39;00m

[36mCell[39m[36m [39m[32mIn[9][39m[32m, line 2[39m, in
[36mdivision[39m[34m(zaehler, nenner)[39m
[32m      1[39m [38;5;28;01mdef[39;00m[38;5;250m
[39m[34mdivision[39m(zaehler, nenner):
[32m----> [39m[32m2[39m     [38;5;28;01mreturn[39;00m
[43mzaehler[49m[43m [49m[43m/[49m[43m [49m[43mnenner[49m

[31mZeroDivisionError[39m: division by zero
```

Auch der Datentyp-Test ist nicht erfolgreich, weil wir ja definiert haben, dass bei falschen Eingabedatentypen ein `ValueError` erzeugt werden soll, wir aber einen `TypeError` erhalten.

```
test_datentyp()
```

```
AssertionError:
[31m-------------------------------------------------------------------------------[39m
[31mTypeError[39m                                 Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[7][39m[32m, line 4[39m, in
[36mtest_datentyp[39m[34m()[39m
[32m      3[39m [38;5;28;01mtry[39;00m:
[32m----> [39m[32m4[39m
[43mdivision[49m[43m([49m[33;43m"[39;49m[33;43mfalscher
typ[39;49m[33;43m"[39;49m[43m,[49m[43m
[49m[32;43m1[39;49m[43m)[49m
[32m      5[39m [38;5;28;01mexcept[39;00m
[38;5;167;01mException[39;00m [38;5;28;01mas[39;00m e:
```

```
[36mCell[39m[36m [39m[32mIn[9][39m[32m, line 2[39m, in
[36mdivision[39m[34m(zaehler, nenner)[39m
[32m      1[39m [38;5;28;01mdef[39;00m[38;5;250m
[39m[34mdivision[39m(zaehler, nenner):
[32m----> [39m[32m2[39m     [38;5;28;01mreturn[39;00m
[43mzaehler[49m[43m [49m[43m/[49m[43m [49m[43mnenner[49m

[31mTypeError[39m: unsupported operand type(s) for /: 'str' and 'int'

During handling of the above exception, another exception occurred:

[31mAssertionError[39m                            Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[12][39m[32m, line 1[39m
[32m----> [39m[32m1[39m [43mtest_datentyp[49m[43m([49m[43m)[49m
[49m

[36mCell[39m[36m [39m[32mIn[7][39m[32m, line 6[39m, in
[36mtest_datentyp[39m[34m()[39m
[32m      4[39m     division([33m"[39m[33mfalscher
typ[39m[33m"[39m, [32m1[39m)
[32m      5[39m [38;5;28;01mexcept[39;00m
[38;5;167;01mException[39;00m [38;5;28;01mas[39;00m e:
[32m----> [39m[32m6[39m     [38;5;28;01massert[39;00m
[38;5;28misinstance[39m(e, [38;5;167;01mValueError[39;00m)
[32m      7[39m [38;5;28;01melse[39;00m:
[32m      8[39m     [38;5;28;01mraise[39;00m
[38;5;167;01mAssertionError[39;00m([33m"[39m[33mDatatype[39m[33m"[39m)

[31mAssertionError[39m:
```

Quiz

Bibliographie